

WEEK 9: LINEAR PROGRAMMING I — MODELLING AND GEOMETRY

Advanced Algorithms

ROADMAP

1. What is a linear program?
2. Three old friends as LPs — max flow, matching, vertex cover
3. Geometry — the feasible region as a polytope
4. Relaxations and the integrality gap
5. Integral polytopes — total unimodularity and bipartite graphs
6. Kruskal and the forest polytope

One idea for the whole session: many problems we already solved fit in one framework.

PART 1: WHAT IS A LINEAR PROGRAM?

WHERE WE'VE BEEN

- Weeks 1–3: **dynamic programming** — solve by decomposing into subproblems
- Weeks 4–6: **greedy & matroids, MST** — solve by local choices with exchange arguments
- Weeks 7–8: **min cut & max flow** — solve by augmentation, certify by cuts

WHERE THIS GOES

Each family came with its own tricks. **Linear programming** is a single framework that can express flow, matching and cover problems — *exactly* for some, as a *relaxation* (a bound) for others — and a single solver handles every LP.

THIS WEEK Learn to *write* problems as LPs and to see what an LP looks like geometrically. Next time: duality, the theory that explains our Week 8 certificates.

THE THREE INGREDIENTS

A linear program (LP) is built from:

- **Decision variables** $x_1, \dots, x_n \in \mathbb{R}$ — the numbers we get to choose
- A **linear objective** to maximise or minimise:
$$c_1x_1 + \dots + c_nx_n$$
- **Linear constraints:** each is
$$a_1x_1 + \dots + a_nx_n \{ \leq, =, \geq \} b$$

THE THREE INGREDIENTS

Example with concrete numbers (we will reuse it all session):

$$\begin{array}{ll}\max & 3x + 2y \\ \text{s.t.} & x \leq 6, \quad x + y \leq 8, \quad x + 3y \leq 18 \\ & x \geq 0, \quad y \geq 0\end{array}$$

Story: a workshop makes x chairs (profit 3) and y tables (profit 2). Only chairs need the machine, and it can finish at most 6 of them; each piece uses one unit of wood (8 available); a table takes three times as much varnish as a chair (18 available).

WHAT "LINEAR" ALLOWS — AND FORBIDS

Allowed

- $2x_1 - 5x_2 + x_3 \leq 7$
- $\sum_{e:v \in e} x_e \leq 1$
- fractional values: $x = \frac{1}{2}$ is fine

Not allowed

- $x_1 x_2 \leq 3$ (products of variables)
- $x^2, |x|, \max(x, y)$ in constraints
- $x \in \{0, 1\}$ (integrality!)

WHAT "LINEAR" ALLOWS — AND FORBIDS

CAREFUL Requiring $x \in \{0, 1\}$ or $x \in \mathbb{Z}$ makes it an *integer* linear program (ILP) — a different beast. ILP is NP-hard (Week 11 will make that precise); LP is solvable in polynomial time. The gap between them is the story of Parts 4 and 5.

STANDARD FORM

Collect the coefficients into a matrix and vectors: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$.

DEFINITION An LP in **standard form** is

$$\max \{ c^\top x : Ax \leq b, x \geq 0 \}$$

n variables, m inequality constraints, all variables nonnegative.

STANDARD FORM

Our example, in matrix clothing:

$$c = \begin{pmatrix} 3 \\ 2 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 6 \\ 8 \\ 18 \end{pmatrix}$$

Row i of A paired with b_i is one constraint: row 2 says $1 \cdot x + 1 \cdot y \leq 8$.

EVERY LP CAN BE PUT IN STANDARD FORM

Standard form looks restrictive. It isn't — four mechanical rewrites:

YOU HAVE

REWRITE AS

$\min c^\top x$	$\max (-c)^\top x$ (negate the answer back)
-----------------	---

$a^\top x \geq \beta$	$(-a)^\top x \leq -\beta$
-----------------------	---------------------------

EVERY LP CAN BE PUT IN STANDARD FORM

YOU HAVE

REWRITE AS

$$a^\top x = \beta$$

$$a^\top x \leq \beta \text{ and } (-a)^\top x \leq -\beta$$

x_j **unrestricted**

$$x_j = x_j^+ - x_j^- \text{ with } x_j^+, x_j^- \geq 0$$

So standard form loses nothing: theorems proved and solvers written for this one shape apply to every LP.

CONVERSION: A WORKED EXAMPLE

Given (x_2 unrestricted):

$$\begin{array}{ll}\min & 2x_1 - x_2 \\ \text{s.t.} & x_1 + x_2 = 4 \\ & x_1 - 3x_2 \geq -6 \\ & x_1 \geq 0\end{array}$$

Standard form (sub $x_2 = u - v$):

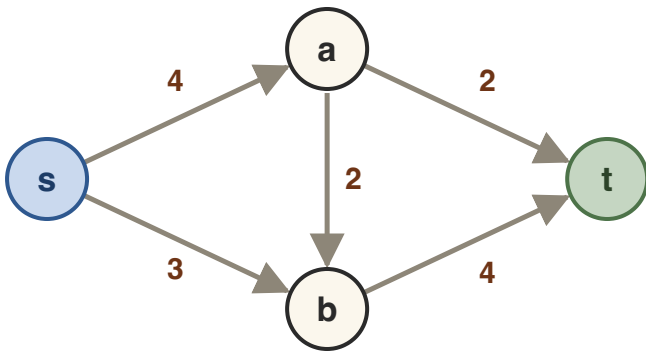
$$\begin{array}{ll}\max & -2x_1 + u - v \\ \text{s.t.} & x_1 + u - v \leq 4 \\ & -x_1 - u + v \leq -4 \\ & -x_1 + 3u - 3v \leq 6 \\ & x_1, u, v \geq 0\end{array}$$

Sanity check: $(x_1, x_2) = (3, 1)$, i.e. $(x_1, u, v) = (3, 1, 0)$, is feasible in both, objective 5 on the left = -5 on the right (sign flipped by the $\min \rightarrow \max$ swap). ✓

PART 2: THREE OLD FRIENDS AS LPs

max flow · bipartite matching · vertex cover

MAXIMUM FLOW



Edge labels are capacities c_e .

One variable per edge: $f_e =$
flow on e .

Five edges, so five numbers to
choose: $f_{sa}, f_{sb}, f_{ab}, f_{at}, f_{bt}$.

MAXIMUM FLOW

A flow must obey two kinds of rules — and both are *linear*:

- **Capacity:** $0 \leq f_e \leq c_e$
- **Conservation:** flow in = flow out at every $v \neq s, t$

Objective: push as much out of s as possible.

MAX FLOW: THE LP

In general $\delta^+(v)$ = edges out of v , $\delta^-(v)$ = edges into v :

$$\begin{array}{ll}\max & \sum_{e \in \delta^+(s)} f_e \\ \text{s.t.} & \sum_{e \in \delta^-(v)} f_e = \sum_{e \in \delta^+(v)} f_e \quad \forall v \neq s, t \\ & 0 \leq f_e \leq c_e \quad \forall e \in E\end{array}$$

MAX FLOW: THE LP

For our graph (5 variables):

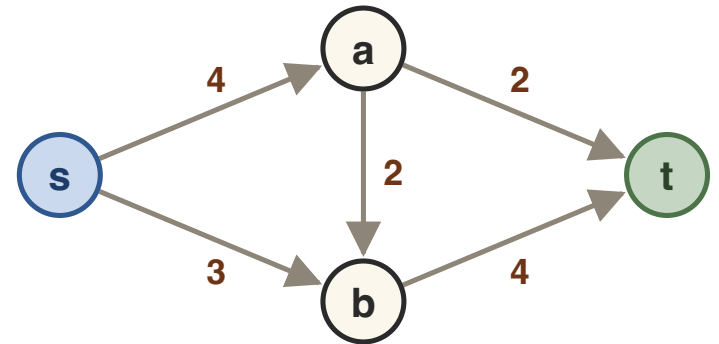
$$\max \quad f_{sa} + f_{sb}$$

$$\text{s.t.} \quad f_{sa} = f_{ab} + f_{at}$$

$$f_{sb} + f_{ab} = f_{bt}$$

$$0 \leq f_{sa} \leq 4, \quad 0 \leq f_{sb} \leq 3$$

$$0 \leq f_{ab} \leq 2, \quad 0 \leq f_{at} \leq 2, \quad 0 \leq f_{bt} \leq 4$$

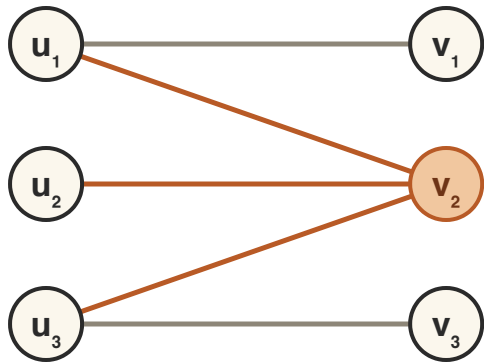


MAX FLOW: THE LP

Optimal value 6: e.g. $f_{sa} = 4$, $f_{at} = 2$, $f_{ab} = 2$, $f_{sb} = 2$, $f_{bt} = 4$.

Matches the min cut $\{s, a, b\}$ vs $\{t\}$ of capacity $2 + 4 = 6$ — Week 8's certificate, now a statement about two LPs (wait for Week 10!).

BIPARTITE MATCHING



One variable per edge: $x_e \in [0, 1]$,
"how much" e is in the matching.

$$\begin{aligned} \max \quad & \sum_{e \in E} x_e \\ \text{s.t.} \quad & \sum_{e: v \in e} x_e \leq 1 \quad \forall v \in V \\ & 0 \leq x_e \leq 1 \quad \forall e \in E \end{aligned}$$

Vertex v_2 and its three incident edges: one degree constraint.

BIPARTITE MATCHING

At v_2 : $x_{u_1v_2} + x_{u_2v_2} + x_{u_3v_2} \leq 1$.

A *matching* is exactly a feasible x with $x_e \in \{0, 1\}$ — the LP is the **relaxation** that lets edges be picked fractionally.

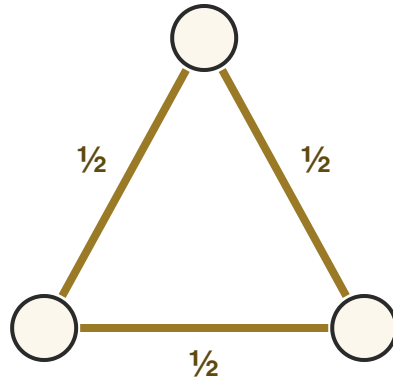
LP VS. MAXIMUM MATCHING

Every matching is a feasible LP solution: set $x_e = 1$ on its edges and 0 elsewhere. So

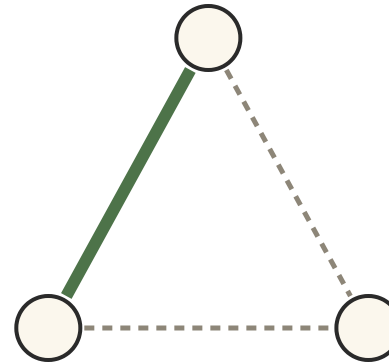
LP value $\geq$ maximum matching size.

Can the LP be strictly larger — can fractions beat every genuine matching?

MATCHING ON THE TRIANGLE



LP: every $x_e = \frac{1}{2}$, value $\frac{3}{2}$



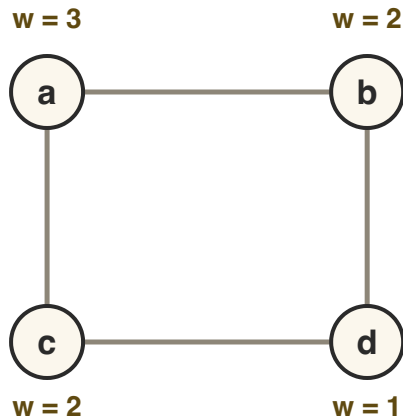
best matching: 1 edge

- All-halves is feasible: each vertex touches two edges, load $\frac{1}{2} + \frac{1}{2} = 1 \leq 1$. ✓ Value $\frac{3}{2}$.
- But any two edges of a triangle share a vertex, so every matching has 1 edge. The fractions win: $\frac{3}{2} > 1$.

BIPARTITE GRAPHS

FACT For *bipartite* graphs, nothing is lost: the matching LP always has an integral optimal solution — its optimum equals the maximum matching size. We will see why later today.

VERTEX COVER

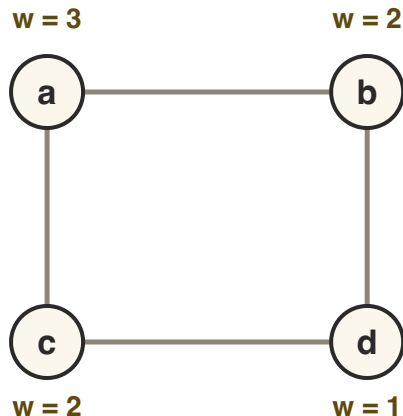


One variable per vertex: $x_v \in [0, 1]$, "how much" v is in the cover.

$$\begin{array}{ll}\min & \sum_v w_v x_v \\ \text{s.t.} & x_u + x_v \geq 1 \quad \forall \{u, v\} \in E \\ & 0 \leq x_v \leq 1 \quad \forall v \in V\end{array}$$

Pick vertices to touch every edge,
minimising total weight.

VERTEX COVER



Pick vertices to touch every edge,
minimising total weight.

Here:

$$\min 3x_a + 2x_b + 2x_c + x_d \text{ with}$$
$$x_a + x_b \geq 1, x_a + x_c \geq 1,$$
$$x_b + x_d \geq 1, x_c + x_d \geq 1.$$

Both $\{a, d\}$ and $\{b, c\}$ cost 4 —
and the LP optimum is also 4 here.

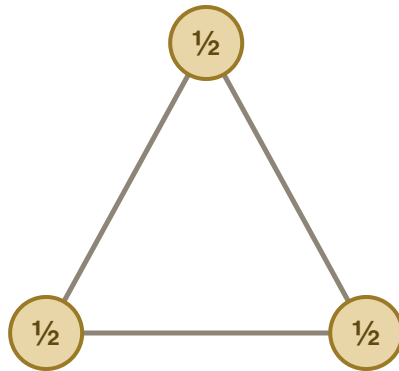
LP VS. MINIMUM VERTEX COVER

Every vertex cover is a feasible LP solution: set $x_v = 1$ on its vertices and 0 elsewhere. So

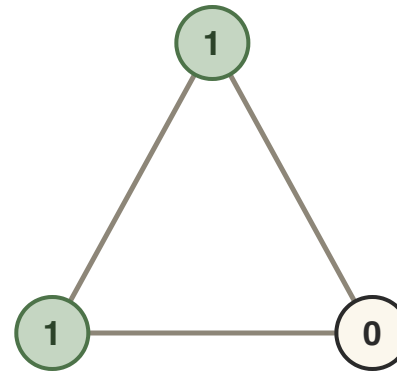
$$\text{LP value} \leq \text{minimum cover weight.}$$

Can the LP be strictly smaller — can fractions undercut every genuine cover?

VERTEX COVER ON THE TRIANGLE



LP: every $x_v = \frac{1}{2}$, cost $\frac{3}{2}$



best integer cover: 2 vertices, cost 2

- All weights 1. All-halves is feasible: every edge gets $\frac{1}{2} + \frac{1}{2} = 1$. So $\text{OPT}_{\text{LP}} \leq \frac{3}{2}$ — in fact $= \frac{3}{2}$: summing the three edge constraints gives $2(x_A + x_B + x_C) \geq 3$.
- Any integral cover needs 2 of the 3 vertices (one vertex misses the opposite edge). So $\text{OPT}_{\text{LP}} = \frac{3}{2} < 2 = \text{OPT}_{\text{ILP}}$.

THE MODELLING PATTERN

PROBLEM	VARIABLES	CONSTRAINTS SAY
max flow	f_e per edge	capacities, conservation
matching	x_e per edge	each vertex used ≤ 1 time
vertex cover	x_v per vertex	each edge covered ≥ 1 time

- Choose variables = "the decisions", one number each.
- Every combinatorial rule became a *linear* inequality or equation.

THE MODELLING PATTERN

- Matching and vertex cover needed a **relaxation**:
 $\{0, 1\} \rightsquigarrow [0, 1]$. Max flow didn't — flows were already fractional-friendly.

Now: what does the set of feasible solutions *look like*?

PART 3: THE GEOMETRY OF AN LP

two variables, so we can draw everything

OUR RUNNING EXAMPLE

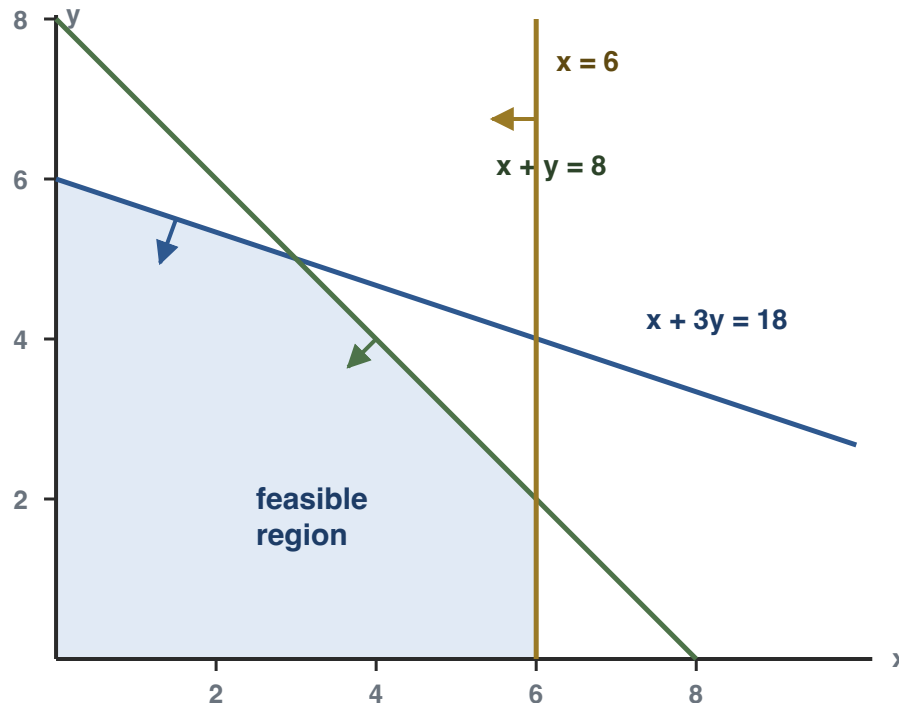
$$\begin{array}{ll} \max & 3x + 2y \\ \text{s.t.} & x \leq 6 \quad (\text{machine}) \\ & x + y \leq 8 \quad (\text{wood}) \\ & x + 3y \leq 18 \quad (\text{varnish}) \\ & x, y \geq 0 \end{array}$$

Two variables $\Rightarrow$ every solution is a point (x, y) in the plane.

OUR RUNNING EXAMPLE

- Each constraint $a_1x + a_2y \leq \beta$ keeps a **half-plane**: the line $a_1x + a_2y = \beta$ plus everything on one side of it.
- Feasible set = intersection of all five half-planes.

BUILDING THE FEASIBLE REGION



$x \geq 0, y \geq 0$: stay in the first quadrant.

- varnish: below $x + 3y = 18$
- wood: below $x + y = 8$
- machine: left of $x = 6$

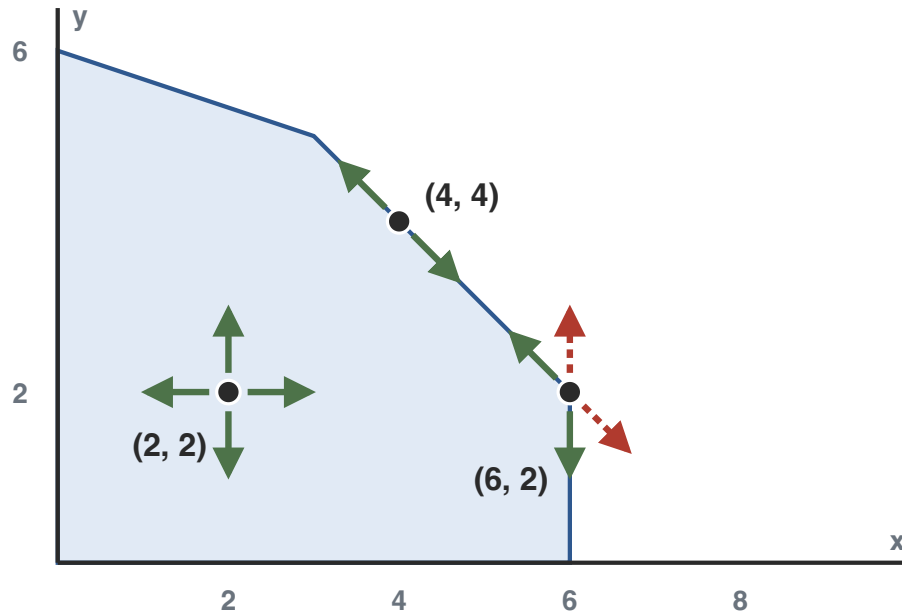
The intersection is a convex polygon — every feasible production plan lives inside it.

POLYTOPES AND VERTICES

DEFINITION A **polyhedron** is an intersection of finitely many half-spaces, $\{x \in \mathbb{R}^n : Ax \leq b\}$. A bounded polyhedron is a **polytope**.

DEFINITION A point z of a polyhedron P is a **vertex** (extreme point) if it is not the midpoint of any segment inside P : there is no $d \neq 0$ with both $z + d \in P$ and $z - d \in P$.

NO TWO-WAY WIGGLE ROOM



Can z step to *both* $z + d$ and $z - d$?

Inside: every direction works both ways. Not a vertex.

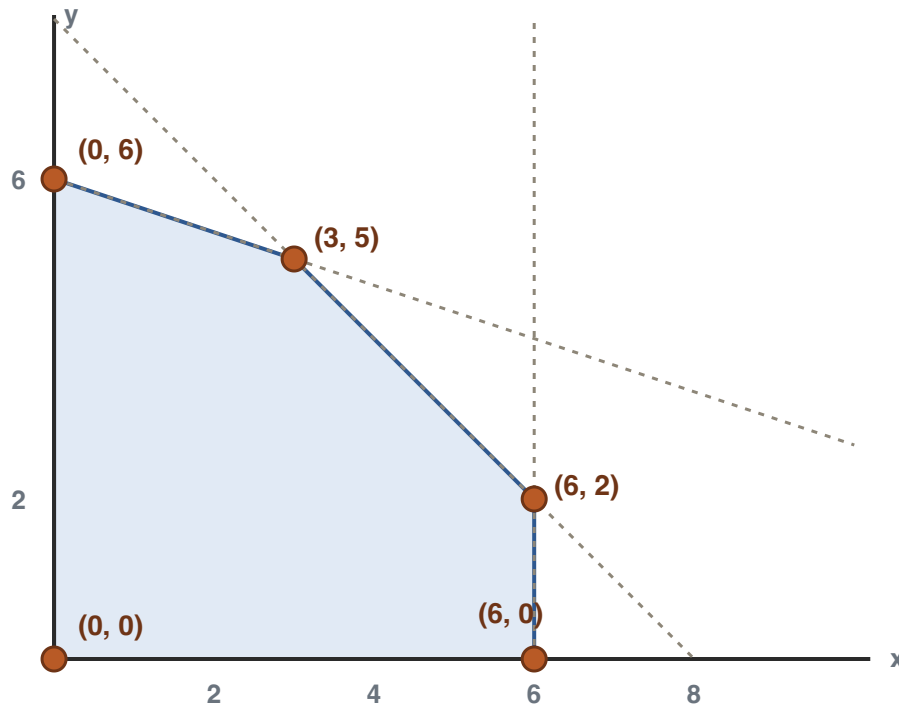
On an edge: slide along it both ways. One such d is enough — not a vertex.

Corner: every d has one side (red) leaving the region. A vertex.

POLYTOPES AND VERTICES

- Equivalent, more computational view in $\mathbb{R}^2$: a vertex is where 2 constraint lines meet (in $\mathbb{R}^n$: n constraints hold with equality, with linearly independent rows) — and the point is feasible.
- Our region is a polytope with 5 vertices. Let's find them all.

THE FIVE VERTICES



Each vertex = two tight constraints:

- $(0, 0)$: $x = 0, y = 0$
- $(6, 0)$: $x = 6, y = 0$
- $(6, 2)$: $x = 6,$
 $x + y = 8$
- $(3, 5)$: $x + y = 8,$
 $x + 3y = 18$
- $(0, 6)$: $x = 0,$
 $x + 3y = 18$

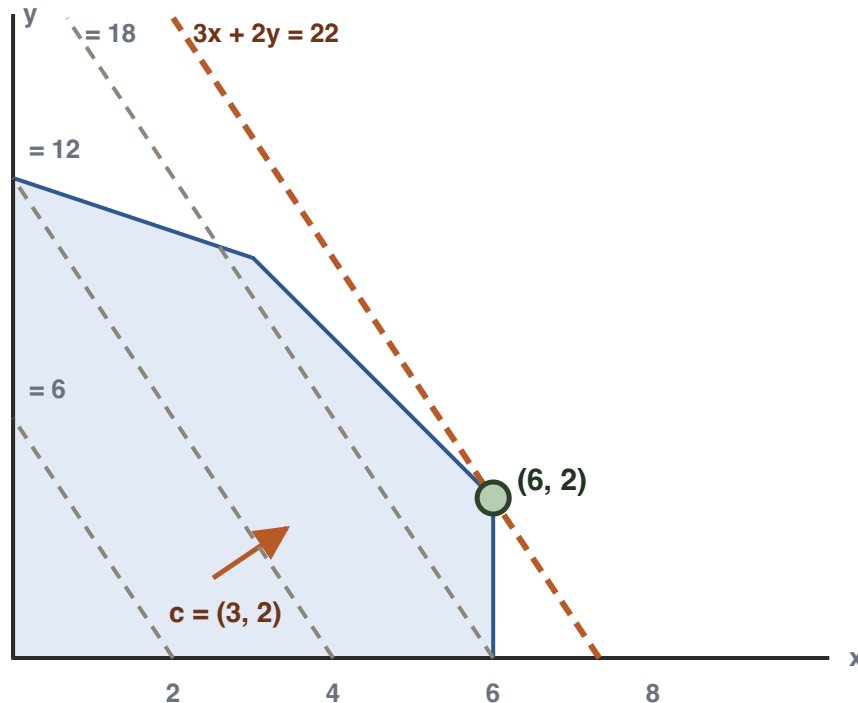
THE FIVE VERTICES

How do we compute a vertex? Solve the 2×2 linear system of its two tight constraints.

e.g. solving $x + y = 8$ and $x + 3y = 18$: subtract to get $2y = 10$, so $y = 5$ — the vertex $(3, 5)$.

SWEEPING THE OBJECTIVE

Every level set $3x + 2y = k$ is a line perpendicular to $c = (3, 2)$. Increasing k slides the line in direction c .



$k = 6$: cuts through the region

$k = 12$: still inside...

$k = 18$: getting tight...

$k = 22$: touches the region at exactly one point — the vertex $(6, 2)$. Push further and we leave the region entirely.

READING OFF THE ANSWER

Cross-check by evaluating $3x + 2y$ at all five vertices:

VERTEX	TIGHT CONSTRAINTS	$3x + 2y$
$(0, 0)$	$x = 0, y = 0$	0
$(6, 0)$	$x = 6, y = 0$	18
$(6, 2)$	$x = 6, x + y = 8$	22
$(3, 5)$	$x + y = 8, x + 3y = 18$	19
$(0, 6)$	$x = 0, x + 3y = 18$	12

READING OFF THE ANSWER

The last level line touching the region pins down the optimum — and the vertex table agrees.

Optimum 22 at (6, 2): make 6 chairs, 2 tables; the machine is at capacity and the wood is used up, and $6 + 3 \cdot 2 = 12 < 18$ varnish left slack.

WHY IS THE OPTIMUM AT A VERTEX?

Keep walking until you can't. Start at any feasible z .

- If z is not a vertex, some $d \neq 0$ has $z \pm d$ feasible. Pick the sign with $c^\top d \geq 0$ and walk until a new constraint becomes tight. The objective never drops.
- Repeat. In 2D: slide across the region to an edge, then along the edge to a corner.

WHY IS THE OPTIMUM AT A VERTEX?

Why the walk ends at a vertex within n steps:

- Constraints tight at z stay tight: each has $a^\top d = 0$, or $z + d$ or $z - d$ would break it.
- The new tight constraint has $a^\top d \neq 0$, so it is independent of the old ones.
- Each step adds an independent tight constraint — after at most n steps, we are at a vertex.

CAN THE WALK GET STUCK?

But does the walk always reach a new constraint? Two cases.

CASE 1: $c^\top d \neq 0$ Pick the sign with $c^\top d > 0$ and walk.
Either a constraint stops us — progress — or we walk forever with $c^\top x \rightarrow \infty$: the LP is **unbounded**.

CAN THE WALK GET STUCK?

CASE 2: $c^\top d = 0$ The objective is flat, so either sign will do. But what if *neither* direction ever hits a constraint?

- Then the whole line $\{z + t d : t \in \mathbb{R}\}$ lies inside the region.
- But some $d_j \neq 0$, so $z_j + t d_j < 0$ once t goes far enough one way — breaking $x_j \geq 0$.
- So one of $\pm d$ does hit a constraint.

CAN THE WALK GET STUCK?

So the walk only fails if the region is empty (nowhere to start) or the LP is unbounded (Case 1 runs forever). That's the fine print in the theorem.

CAREFUL Case 2 needed $x \geq 0$. Without it, the strip $\{(x, y) : 0 \leq y \leq 1\}$ with $\max y$ has optimum 1 — but no vertices at all.

THE FUNDAMENTAL THEOREM

FUNDAMENTAL THEOREM OF LP For an LP $\max\{c^\top x : Ax \leq b, x \geq 0\}$, exactly one of the following holds:

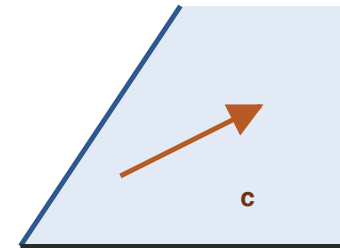
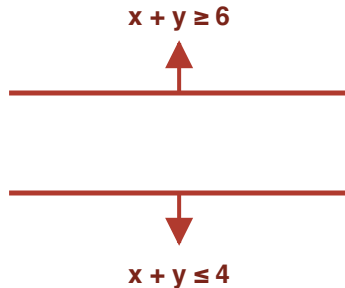
1. the LP is **infeasible** (the region is empty), or
2. the LP is **unbounded** (feasible points of arbitrarily large value exist), or
3. the LP has a finite optimum, and some vertex of the feasible region is optimal.

THE FUNDAMENTAL THEOREM

Consequence: an LP — a continuous problem over infinitely many points — is secretly a *finite* problem: only the vertices matter.

(Ties are allowed: if c is perpendicular to an edge, the whole edge is optimal — e.g. objective $x + y$ in our example is 8 along the entire segment from $(6, 2)$ to $(3, 5)$. A vertex is still among the optima.)

THE OTHER TWO OUTCOMES, DRAWN



Infeasible: half-planes with empty intersection. No solutions at all.

Unbounded: the region is open in direction c — walk forever, value $\rightarrow \infty$.

CAREFUL "Unbounded *region*" $\neq$ "unbounded *LP*": if c points away from the open end, the optimum is still finite and still at a vertex. It's the objective that has to escape.

AN ALGORITHM FOR TINY LPs: VERTEX ENUMERATION

The fundamental theorem is an algorithm — check every vertex.
With $n = 2$ variables and m constraints (counting $x \geq 0, y \geq 0$):

```
best ←  $-\infty$ ,  argbest ← none
for each pair {i, j} of constraints:           //  $O(m^2)$  candidate corners
    solve  $a_i \cdot (x,y) = b_i$  and  $a_j \cdot (x,y) = b_j$  // a  $2 \times 2$  linear system
    if the lines are parallel: continue
    if (x,y) satisfies ALL m constraints:       // feasibility check,  $O(m)$ 
        if  $c \cdot (x,y) > \text{best}$ : best ←  $c \cdot (x,y)$ , argbest ← (x,y)
return argbest, best                          // none found  $\Leftrightarrow$  infeasible
```

AN ALGORITHM FOR TINY LPs: VERTEX ENUMERATION

- **Caveat:** on an *unbounded* LP this still returns the best vertex that exists — a confidently finite, wrong answer.
Enumeration assumes a bounded optimum.
- Running time $O(m^3)$ — fine for $m \leq 100$, hopeless in general: in $\mathbb{R}^n$ there can be $\binom{m}{n}$ candidates, exponentially many.

WHAT'S USED IN PRACTICE

ALGORITHM	IDEA	STATUS
Simplex (Dantzig, 1947)	walk vertex → better neighbouring vertex until stuck	exponential in the worst case, excellent in practice
Ellipsoid (Khachiyan, 1979)	shrink an ellipsoid around the feasible region	first polynomial-time algorithm; mostly theoretical

WHAT'S USED IN PRACTICE

ALGORITHM	IDEA	STATUS
Interior point (Karmarkar, 1984)	move through the interior of the polytope, not along its edges	polynomial <i>and</i> practical

WHAT'S USED IN PRACTICE

FACT Linear programming is solvable in **polynomial time** (in the bit-size of the input). So from now on: if a problem can be written *exactly* as an LP of *polynomial size* — polynomially many variables and constraints — then we can solve it efficiently.

PART 4: RELAXATIONS AND THE INTEGRALITY GAP

THE LP AS A BOUND

Part 2 showed both directions:

- Minimisation (vertex cover): $\text{OPT}_{\text{LP}} \leq \text{OPT}_{\text{ILP}}$ — the LP is an efficiently computable **lower bound**.
- Maximisation (matching): $\text{OPT}_{\text{LP}} \geq \text{OPT}_{\text{ILP}}$ — an **upper bound**.

Question: how far apart can they be? If never far, the LP is a great proxy — the engine behind Week 12's approximation algorithms.

INTEGRALITY GAP

DEFINITION The **integrality gap** of an LP relaxation (of a minimisation problem) is the smallest α such that

$$\text{OPT}_{\text{ILP}} \leq \alpha \cdot \text{OPT}_{\text{LP}} \quad \text{on every instance}$$

— the worst-case factor by which the true optimum exceeds the LP bound.

Gap = 1 means the relaxation is **exact**: the LP always has an integral optimum.

VERTEX COVER GAP

- The triangle shows the gap is $\geq \frac{2}{3/2} = \frac{4}{3}$.
- The complete graph K_n pushes it to $\frac{n-1}{n/2} \rightarrow 2$.

Can it be worse than 2? No — and the proof is short.

VERTEX COVER GAP ≤ 2

Take an optimal LP solution x^* and round: $C = \{ v : x_v^* \geq \frac{1}{2} \}$.

- **C is a cover.** Every edge has $x_u^* + x_v^* \geq 1$, so one endpoint has $x^* \geq \frac{1}{2}$ and lands in C .
- **C costs at most twice the LP.**

$$\begin{aligned} w(C) &= \sum_{v \in C} w_v \leq \sum_{v \in C} w_v \cdot 2x_v^* \\ &\leq 2 \sum_v w_v x_v^* = 2 \text{OPT}_{\text{LP}} \end{aligned}$$

(each $v \in C$ has $2x_v^* \geq 1$; then add the $v \notin C$ terms, all ≥ 0)

VERTEX COVER GAP ≤ 2

C is an integral cover, so on every instance

$$\text{OPT}_{\text{ILP}} \leq w(C) \leq 2 \text{OPT}_{\text{LP}}.$$

The gap is at most 2 — and K_n shows it's exactly 2.

Bonus: solve the LP, round — a polynomial-time cover within $2 \times$ optimal. Week 12 returns to this as an *approximation algorithm*.

PART 5: INTEGRAL POLYTOPES AND TOTAL UNIMODULARITY

GEOMETRY OF THE GAP

The dictionary between Parts 3 and 4:

- The LP optimum lives at a **vertex of the relaxation's polytope**.
- If every vertex is a 0/1 point (an *integral polytope*), solving the LP *is* solving the combinatorial problem.

GEOMETRY OF THE GAP

- The triangle's cover polytope has the genuinely fractional vertex $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ — a corner that no integer point can imitate. Gaps are *geometric defects*: fractional corners.

So a natural question: which polytopes from combinatorics have *all* corners integral? Two answers today: one from linear algebra (next), and one from Week 5 (Part 6).

BIPARTITE GRAPHS: GAP 1

FACT On *bipartite* graphs, both the matching LP and the vertex cover LP have integrality gap 1: every vertex of their polytopes is integral. Both triangle examples hinge on an odd cycle — bipartite graphs have none. Next time, combined with duality, this gives König's theorem.

TOTALLY UNIMODULAR

DEFINITION A matrix A is **totally unimodular (TU)** if every square submatrix has determinant 0, $+1$ or -1 .

THEOREM If A is TU and b is integral, every vertex of $\{x : Ax \leq b, x \geq 0\}$ is integral.

CRAMER'S RULE

For a square system $Bx = b'$ with $\det B \neq 0$:

$$x_j = \frac{\det B_j}{\det B}$$

where B_j is B with **column j replaced by b'** .

Example: the vertex $(3, 5)$ of our pentagon, where $x + y = 8$ and $x + 3y = 18$ are tight:

$$B = \begin{pmatrix} 1 & 1 \\ 1 & 3 \end{pmatrix}, \quad b' = \begin{pmatrix} 8 \\ 18 \end{pmatrix}, \quad \det B = 2.$$

CRAMER'S RULE

Replace one column of B by b' at a time:

$$x = \frac{\det \begin{pmatrix} 8 & 1 \\ 18 & 3 \end{pmatrix}}{2} = \frac{24 - 18}{2} = 3$$

$$y = \frac{\det \begin{pmatrix} 1 & 8 \\ 1 & 18 \end{pmatrix}}{2} = \frac{18 - 8}{2} = 5$$

The only division is by $\det B$. (Here $\det B = 2$, and the numerators happened to be even.)

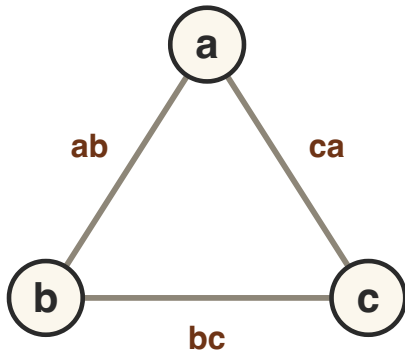
WHY TU GIVES INTEGERS

- Part 3: a vertex is where n linearly independent constraints are tight. So it solves $Bx = b'$ for a nonsingular square B built from rows of the constraints, with b' integral.
- Cramer's rule: $x_j = \frac{\det B_j}{\det B}$. The numerator is an integer.
- The denominator is ± 1 : rows from $x \geq 0$ are unit vectors, and expanding along them leaves a square submatrix of A .

So every coordinate of every vertex is an integer.

THE INCIDENCE MATRIX

The **vertex-edge incidence matrix** A of a graph has one row per vertex and one column per edge: $A_{v,e} = 1$ if v is an endpoint of e , else 0.



	ab	bc	ca
a	1	0	1
b	1	1	0
c	0	1	1

Every column has exactly two 1s: the edge's two endpoints.

THE INCIDENCE MATRIX

Both of our LPs are built from it:

- **Matching:** row v says $\sum_{e:v \in e} x_e \leq 1$. All together: $Ax \leq \mathbf{1}$.
- **Vertex cover:** column $e = uv$ says $x_u + x_v \geq 1$. All together: $A^\top x \geq \mathbf{1}$.

So whether their polytopes have integral corners depends on the determinants of A 's square submatrices.

THE TRIANGLE IS NOT TU

The triangle's incidence matrix has determinant 2:

$$\det \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} = 1 \cdot (1 - 0) - 0 + 1 \cdot (1 - 0) = 2.$$

The vertex cover LP's corner has all three edge constraints tight:

$x_a + x_b = x_b + x_c = x_a + x_c = 1$. Cramer divides by 2 — that's where the $\frac{1}{2}$'s come from.

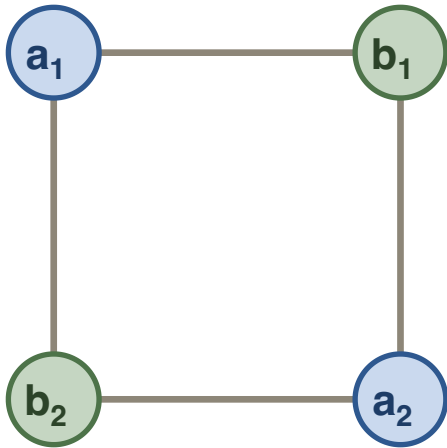
BIPARTITE GRAPHS ARE TU

Claim: the incidence matrix of a bipartite graph (sides L , R) is TU. Induct on the size of a square submatrix B . Each column has at most two 1s — one in an L row, one in an R row.

- A column of zeros: $\det B = 0$.
- A column with one 1: expand along it; $\det B = \pm$ a smaller determinant, which is 0 or ± 1 by induction.
- Every column has two 1s: the L rows and the R rows each sum to all-ones, so $(L \text{ rows}) - (R \text{ rows}) = 0$. The rows are dependent: $\det B = 0$.

BIPARTITE GRAPHS ARE TU

The last case on the 4-cycle: $L = \{a_1, a_2\}$, $R = \{b_1, b_2\}$.



	a_1b_1	b_1a_2	a_2b_2	b_2a_1
a_1	1	0	0	1
a_2	0	1	1	0
b_1	1	1	0	0
b_2	0	0	1	1

$a_1 + a_2 = (1, 1, 1, 1) = b_1 + b_2$, so the rows are dependent:
 $\det = 0$.

BIPARTITE GRAPHS ARE TU

- The matching LP uses A ; the vertex cover LP uses A^T with $\geq$ rows. Transposing and negating keep TU.
- So on bipartite graphs both polytopes have integral corners: gap 1, as the Fact promised.
- The odd cycle breaks exactly the last case: the triangle's rows can't be split into L and R .

PART 6: KRUSKAL AND THE FOREST POLYTOPE

FORESTS AS VECTORS

- Week 5: Kruskal adds edges smallest weight first, skipping any edge that closes a cycle, and returns an MST. Correct because forests form a **matroid**.
- Encode an edge set $F \subseteq E$ as its indicator vector $\mathbb{1}_F \in \{0, 1\}^E$ — one coordinate per edge, like our matching LPs.

THE GOAL

We want linear constraints such that

1. **Valid:** every forest satisfies them, and
2. **No extra corners:** every vertex of the polytope is a forest.

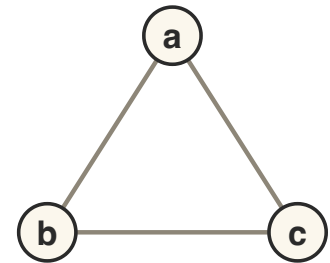
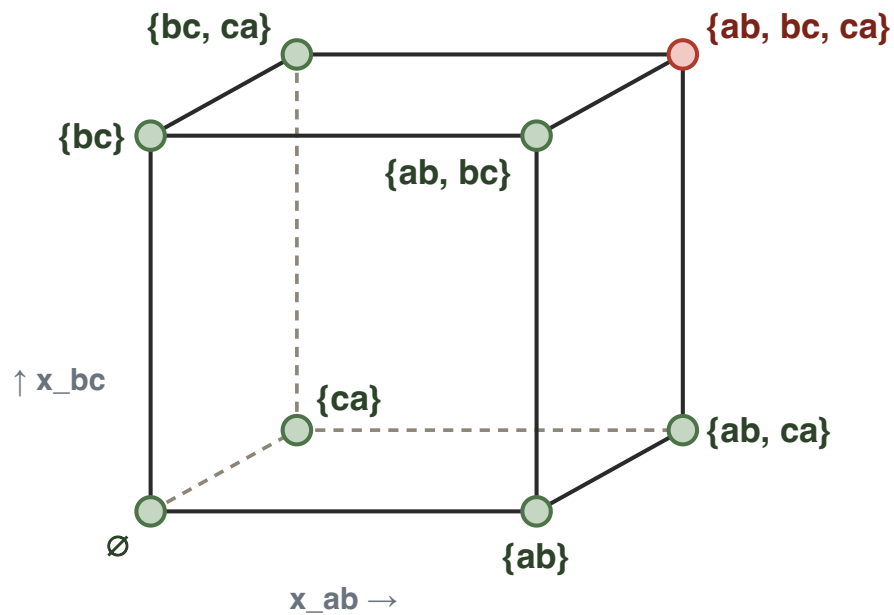
Together: the polytope is exactly the convex hull of the forests.
Then MST is an LP — and we can watch what Kruskal does to it.

THE GOAL

- Checking (1) is easy: count edges.
- To see (2) *fail*: find an objective where some feasible point beats every forest. The LP optimum sits at a vertex — and that vertex can't be a forest.

Build it on the smallest graph with a cycle: the triangle. Three edges, so $x \in \mathbb{R}^3$ — we can draw it.

EDGE SETS OF A TRIANGLE



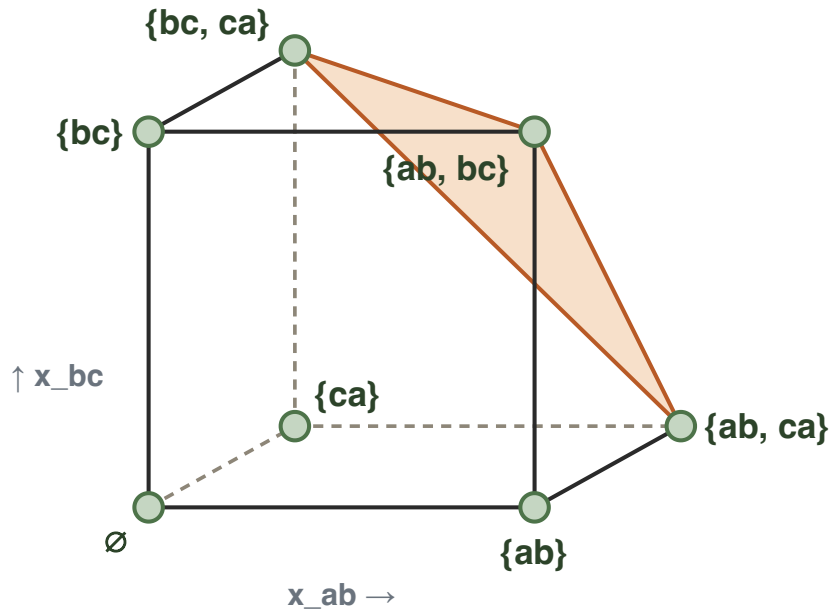
- $2^3 = 8$ edge sets = the 8 corners of the cube.
- 7 are forests. Only $\{ab, bc, ca\}$ — the whole triangle — is a cycle.

FIRST TRY: THE CUBE

- Every forest satisfies $0 \leq x_e \leq 1$, so (1) ✓. Try the cube.
- (2) ✗: maximise $x_{ab} + x_{bc} + x_{ca}$. Every forest scores at most 2, but the LP answers $(1, 1, 1)$ — the cycle, scoring 3.

We need a constraint that keeps (1) and cuts off the cycle.

CUT OFF THE CYCLE



A forest uses at most 2 of the triangle's 3 edges, so this is valid — (1) ✓:

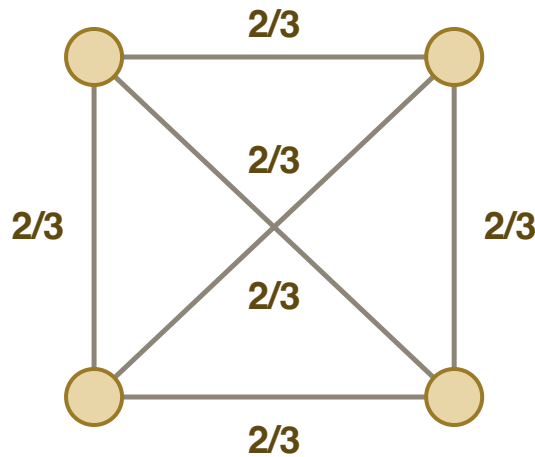
$$x_{ab} + x_{bc} + x_{ca} \leq 2$$

The plane slices off the red corner.

It creates no new corners: it meets the cube only at 3 existing ones.

Corners left: **exactly the 7 forests** — (2) ✓.

ONE CUT PER CYCLE?

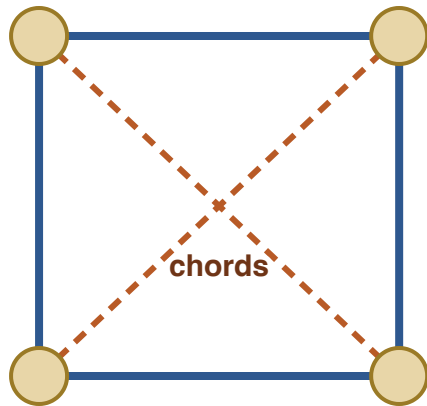


K_4 , every $x_e = \frac{2}{3}$.

Natural guess: for every cycle C ,
 $\sum_{e \in C} x_e \leq |C| - 1$. Valid — (1) ✓.

- Each triangle: $3 \cdot \frac{2}{3} = 2 \leq 2$. ✓
- Each 4-cycle: $4 \cdot \frac{2}{3} = \frac{8}{3} \leq 3$. ✓
- Maximise $\sum_e x_e$: this point scores $6 \cdot \frac{2}{3} = 4$, but a forest on 4 vertices has at most 3 edges. (2) ✗

WHAT THE CYCLES MISS



A 4-cycle of K_4 (blue) and its chords.

The 4-cycle constraint counts only the cycle's own edges: $4 \cdot \frac{2}{3} = \frac{8}{3} \leq 3$.

It ignores the **chords**. But a forest can't use more than 3 edges of *any* kind among these 4 vertices — and all 6 edges give $4 > 3$.

Repair: count **every** edge inside the cycle's vertex set. (The triangle has no chords — which is why it couldn't tell the difference.)

COUNT BY VERTICES

- Week 5: a forest with k edges on s vertices has $s - k$ components.
- At least one component, so $k \leq s - 1$.
- The edges of a forest that lie inside a vertex set S form a forest on S — so these are valid, (1) ✓:

$$\sum_{e \subseteq S} x_e \leq |S| - 1 \quad \forall S \subseteq V, |S| \geq 2$$

($e \subseteq S$: both endpoints of e lie in S .)

COUNT BY VERTICES

Sanity check — it includes every earlier fix:

- $S = \{a, b, c\}$ in the triangle: $x_{ab} + x_{bc} + x_{ca} \leq 2$.
- $S =$ all of K_4 : $\sum_e x_e \leq 3$ — kills the $\frac{2}{3}$ point, whose total is 4.
- $S = \{u, v\}$: the only edge inside is uv , so $x_{uv} \leq 1$ — the cube's upper bounds are special cases.

Does it get (2) in general? That's the theorem.

THE FOREST POLYTOPE

THEOREM (EDMONDS) These constraints — the **subtour elimination constraints** — are enough:

$$P_{\text{forest}} = \left\{ x \in \mathbb{R}^E : x \geq 0, \right. \\ \left. \sum_{e \subseteq S} x_e \leq |S| - 1 \quad \forall S \subseteq V, |S| \geq 2 \right\}$$

is exactly the convex hull of the forests of G . Every vertex is a forest — the polytope is **integral**. (We won't prove this here.)

THE FOREST POLYTOPE

- Integral, but *not* by TU: rows $\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$ of K_4 on columns 12, 13, 14 give determinant -2 . Here integrality comes from the matroid structure.
- Exponentially many constraints — and that's allowed! A polytope doesn't care how many inequalities describe it. (Solving such LPs efficiently needs the ellipsoid method + a "separation oracle" — a story for another course.)
- For *spanning trees* of a connected G : add $\sum_{e \in E} x_e = |V| - 1$; the vertices become exactly the spanning trees.

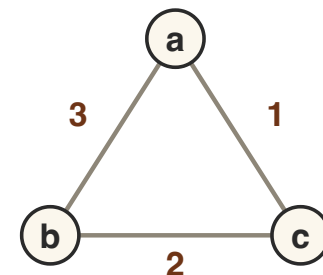
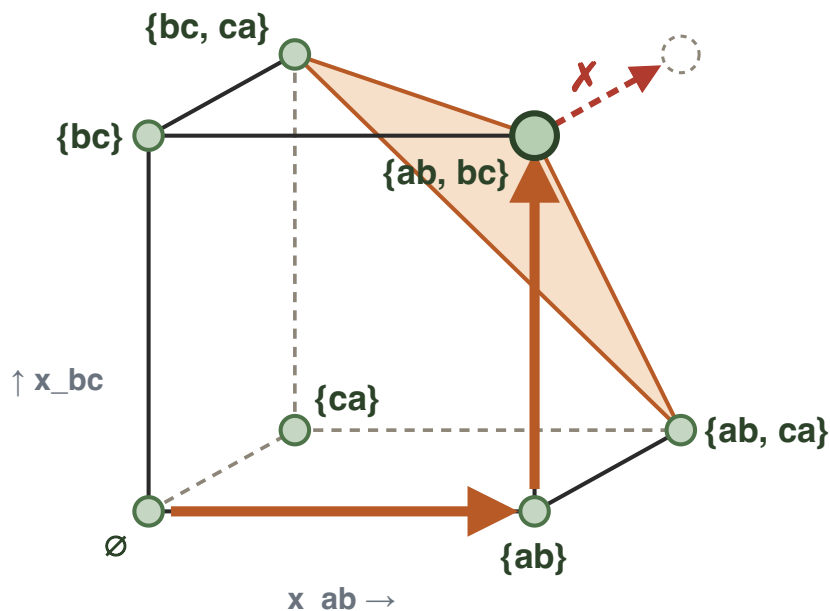
MST AS AN LP

Pick C bigger than every weight and set $w'_e = C - w_e > 0$.

- Every spanning tree has $|V| - 1$ edges, so $w'(T) = (|V| - 1) C - w(T)$: maximising w' is minimising w .
- All $w'_e > 0$, so a best forest never stops short of spanning.

So the MST is the optimal vertex of $\max \{w'^\top x : x \in P_{\text{forest}}\}$.
Largest w' first is smallest w first: Kruskal's order.

KRUSKAL WALKS THE POLYTOPE



labels $w' = 4 - w$; value $= w'^T x$

- Start at $\emptyset$: value 0.
- Take ab : to $\{ab\}$, value 3.
- Take bc : to $\{ab, bc\}$, value 5.
- ca closes a cycle — that corner is cut off. Stop.

WHY IT STOPS AT THE OPTIMUM

Neighbours of $\{ab, bc\}$ on the polytope:

- Drop an edge: $\{ab\}$ (value 3), $\{bc\}$ (value 2).
- Swap one edge for another along the cut face: $\{ab, ca\}$ (value 4), $\{bc, ca\}$ (value 3).

None beats 5. On any LP, a vertex with no better neighbouring vertex is optimal — convexity leaves no local traps (we won't prove this here).

WHY IT STOPS AT THE OPTIMUM

TAKEAWAY Kruskal is a simplex-style walk on P_{forest} : each accepted edge is a step to an adjacent, better vertex.

- Edmonds' theorem makes the polytope integral, so its optimal vertex is a real spanning tree.
- The exchange property is why greedy gets there by only *adding* edges — it never has to back up.

THE BIG PICTURE

WEEK	THEN	NOW, IN LP LANGUAGE
4–5	greedy + exchange	walking the skeleton of an integral polytope
8	max flow, min cut certificate	an LP that a cut certifies (W10: duality)
8	bipartite matching via flow	an LP with integrality gap 1
3, 11, 12	vertex cover arc	an LP with gap $\rightarrow 2$; a 2-approximation in W12

One solver, one geometry — and the special structure of each problem shows up as the *shape of its polytope*.

NEXT TIME: DUALITY

- In Week 4, exchange arguments *certified* that greedy's answer couldn't be beaten. In Week 8, a saturated cut certified a max flow. Both times: a short proof of optimality you can check without rerunning the algorithm.
- Next time we'll see this is no accident: every LP has a shadow LP — its **dual** — whose feasible solutions are certificates bounding the original.

NEXT TIME: DUALITY

- Max-flow/min-cut becomes a two-line consequence of duality (same theorem as Week 8, second proof, wider reach), and König's theorem lands the matching-vs-vertex-cover capstone.

TAKEAWAY This week: write it as an LP, then look at the polytope — the optimum sits at a corner. Next time: how to *prove* a corner is the best one.